

Implementing the Ring Election Algorithm

The **Ring Election Algorithm** is a distributed algorithm used to elect a coordinator or leader among nodes in a network arranged in a logical ring. Each process in the network has a unique identifier, and the algorithm initiates when a node detects the failure of the current leader. The initiating node sends an election message containing its own identifier to the next node in the ring. Each node forwards the election message around the ring, replacing the identifier in the message with its own if it is higher. When the message returns to the initiator, it contains the highest identifier, which becomes the new leader. The new leader is then announced to all nodes in the network. This algorithm ensures that the process with the highest identifier is elected as the coordinator, and it works efficiently in asynchronous systems with minimal message overhead.

1. Process Simulation:

- o Simulates multiple processes with unique IDs.
- o Each process can fail or recover.
- o The leader is automatically elected based on the highest PID.

2. Ring Election Algorithm:

- o When a process detects the failure of the current leader, it sends an "election" message around the ring.
- o Each process appends its PID to the message and forwards it.
- o The process with the highest PID is elected leader after the message circulates back to the initiator.

3. Process Failure and Recovery:

- o Simulates the failure of the current leader.
- o Automatically initiates an election if the leader fails.
- o Allows a recovered process to rejoin the system and trigger a new election if its PID is higher than the current leader.

Compile the Code:

Open your terminal and navigate to the directory containing the RingElection.java file:

```
javac RingElection.java
```

Run the Program:

After successful compilation, run the program using:

```
java RingElection
```

Output:

```
PS D:\SEM-5\Distributed System\lab5> cd "d:\SEM-5\Distributed System\lab5"
Starting election...
Process 5 is elected as the new leader.
Process 5 is still the leader.
Process 5 has failed.
Leader has failed! Starting a new election...

Starting election...
Process 4 is elected as the new leader.
Process 5 has recovered.

Starting election...
Process 5 is elected as the new leader.
```

Architecture:

- *Processes*: Each process is represented by a class instance with a unique PID. They are organized in a ring structure, allowing communication with their immediate neighbour.
- *Election Process*: A new election starts when the current leader fails, and the process with the highest PID in the ring becomes the new leader.
- *Failure & Recovery*: Processes can fail and recover dynamically. When a process with a higher PID than the current leader recovers, it triggers a new election.

Conclusion:

This project successfully simulates the Ring Election Algorithm and handles process failures and recoveries efficiently. The system ensures that a leader is always available, even when the leader fails, and recovered processes are integrated back into the system.